

# FYR

## A CDN-Only Open-Source Web Framework

Reactive HTML, Optional Browser Python, Rust WebAssembly, and Any JSON Backend

| Project Item           | Description                                                                     |
|------------------------|---------------------------------------------------------------------------------|
| Project name           | Fyr (pronounced “fire”)                                                         |
| Project type           | Open-source, CDN-only browser framework                                         |
| Version target         | 0.1.0 prototype, progressing toward 1.0.0                                       |
| Primary implementation | TypeScript/JavaScript with optional Rust WebAssembly                            |
| Optional integrations  | Pyodide, Flask, FastAPI, Django, Node.js, PHP, Rust/Axum                        |
| Target users           | Students, educators, independent developers, small teams, and rapid prototypers |
| Distribution           | Versioned CDN assets through jsDelivr and UNPKG                                 |
| License recommendation | MIT License                                                                     |
| Date                   | July 2026                                                                       |

*Computer Science Project Design, Requirements, Architecture, Governance, and Implementation Plan*

# Table of Contents

## Executive Summary

1. Project Background
2. Problem Statement
3. Proposed Solution
4. Aim, Objectives, and Scope
5. Stakeholders and Target Users
6. Functional Requirements
7. Non-Functional Requirements
8. System Architecture
9. Core Framework Design
10. CDN-Only Distribution Model
11. Optional Browser Python
12. Rust WebAssembly Integration
13. Backend Independence
14. Security and Threat Model
15. Data Structures and Algorithms
16. User Experience and Accessibility
17. Testing and Quality Assurance
18. Performance and Benchmarking
19. Open-Source Governance
20. Repository and File Structure
21. Development Methodology
22. Deployment and Release Strategy
23. Risks, Limitations, and Mitigation
24. Evaluation Criteria
25. Roadmap
26. Conclusion

## Appendices

## Executive Summary

Fyr is an open-source, CDN-only web framework designed to make interactive web application development possible without requiring end users to install Node.js, npm, a project generator, a bundler, or a framework toolchain. Developers add a versioned script tag to ordinary HTML and use readable Fyr directives to bind state, events, forms, conditional rendering, lists, HTTP requests, and server actions. The framework follows progressive enhancement: the smallest application loads only the core browser script, while routing, browser Python, WebAssembly, sockets, and UI helpers remain optional modules.

The key innovation is not merely a new directive syntax. Fyr combines zero-install distribution, reactive HTML, backend independence, optional Python execution through Pyodide, and first-class Rust/WebAssembly loading in one coherent browser API. Applications may use Flask, FastAPI, Django, Express, PHP, Rust/Axum, serverless functions, or any service that communicates through HTTP and JSON. Secure business logic, credentials, authorization decisions, and database access remain on the server.

This report defines the complete computer science project: the problem being solved, requirements, architecture, algorithms, security model, open-source governance, repository structure, testing strategy, deployment model, risk analysis, and roadmap. Fyr is intended to be educational and practical: a student can begin with three files, while advanced developers can extend the same project with routing, authentication, browser computation, WebAssembly, and remote APIs.

**Core value proposition:** Add one CDN script to normal HTML and build a reactive application without installing frontend tooling.

## 1. Project Background

Modern frontend development has become powerful, but its entry path often depends on package managers, build systems, compilers, configuration files, and rapidly changing dependencies. These tools are valuable for large products, yet they can create unnecessary friction for learning exercises, prototypes, internal tools, static hosting, and small applications.

CDN-based libraries such as Alpine.js and Vue global builds prove that meaningful browser interactivity can be delivered through script tags. Fyr extends this idea by combining an HTML-first reactive engine with optional browser Python and Rust/WebAssembly support, while remaining independent of any specific backend framework.

## 2. Problem Statement

Students, educators, independent developers, and small teams need a simpler method for building interactive and full-stack web applications without first installing and configuring a large frontend toolchain. Existing approaches often introduce several connected problems:

- **Installation barrier:** Node.js, npm, framework CLIs, bundlers, and project generators may be required before a page can run.
- **Tooling fragmentation:** frontend state, routing, HTTP calls, forms, Python integration, WebAssembly loading, and backend communication often require unrelated libraries.
- **Learning overload:** beginners must understand multiple configuration systems before understanding browser state and events.
- **Deployment complexity:** simple static applications become build artifacts instead of directly hostable HTML, CSS, and JavaScript files.
- **Language isolation:** Python learners cannot easily apply Python inside browser exercises, while Rust developers must manually create WebAssembly bridges.

- Backend lock-in: some full-stack frameworks force a specific server runtime even when developers already prefer Flask, FastAPI, PHP, Node.js, or Rust.
- Security confusion: developers may incorrectly place secrets or authorization logic in browser code because the trust boundary is not explicit.

**Formal problem:** How can an open-source web framework provide reactive interfaces, optional advanced runtimes, and backend connectivity while remaining zero-install for the framework user?

### 3. Proposed Solution

Fyr provides precompiled, browser-ready files published through a public CDN. Users do not install the framework. They include only the modules needed by their page and write normal HTML, CSS, and JavaScript.

```
<!doctype html>
<html>
<head>
  <script defer src="https://cdn.jsdelivr.net/npm/fyr-framework@0.1.0/dist/fyr.min.js"></script>
  <script defer src="app.js"></script>
</head>
<body>
  <main fyr-controller="counter">
    <strong fyr-text="count"></strong>
    <button fyr-click="count++">Increase</button>
  </main>
</body>
</html>

Fyr.controller("counter", {
  state: { count: 0 }
});
```

The framework creator may use TypeScript, Vite, Rust, tests, and release scripts internally. Those tools are implementation details. Framework users receive only stable assets such as `fyr.min.js`, `fyr-router.min.js`, `fyr-python.min.js`, `fyr-wasm.min.js`, worker files, CSS, and precompiled `.wasm` binaries.

## 4. Aim, Objectives, and Scope

### 4.1 Project Aim

To design and implement an open-source, CDN-only web framework that adds reactive application capabilities to ordinary HTML without mandatory installation or compilation, while supporting optional browser Python, Rust WebAssembly, and any HTTP/JSON backend.

### 4.2 Objectives

- Provide zero-install usage through one or more versioned CDN script tags.
- Implement reactive state using standard browser APIs.
- Support clear HTML directives for text, events, forms, visibility, loops, attributes, classes, styles, and initialization.
- Provide controller, component, lifecycle, plugin, HTTP, storage, and server-action APIs.
- Offer optional routing without requiring a bundler.
- Run Python in the browser only when requested, using Pyodide and a Web Worker.
- Load precompiled Rust/WebAssembly modules through a consistent registry and memory bridge.
- Remain backend-independent and document Flask and FastAPI as reference integrations.
- Publish transparent source code, contribution instructions, tests, benchmarks, security policy, and release history.
- Maintain a small core, pinned versions, reproducible builds, and long-term compatibility.

### 4.3 In Scope

- Browser reactivity and directives
- CDN distribution
- Optional plugins
- HTTP/JSON communication
- Reference backend examples
- Open-source governance
- Testing, security, documentation, and release automation

### 4.4 Out of Scope for Version 0.1

- A visual drag-and-drop builder
- A mandatory backend runtime
- A full virtual DOM
- Server-side rendering
- Automatic native mobile packaging
- A package ecosystem marketplace
- Storing secrets in browser code

## 5. Stakeholders and Target Users

| Stakeholder              | Need                                             | How Fyr responds                                          |
|--------------------------|--------------------------------------------------|-----------------------------------------------------------|
| Students                 | Low setup and understandable code                | Script tag, normal HTML, readable directives, examples    |
| Educators                | Reliable classroom setup                         | No per-device framework installation; pinned CDN versions |
| Independent developers   | Fast prototypes and small tools                  | Three-file projects and optional APIs                     |
| Python developers        | Use familiar backend and optional browser Python | Flask/FastAPI examples plus Pyodide plugin                |
| Rust developers          | Browser performance path                         | Standard WebAssembly loader and module registry           |
| Open-source contributors | Clear ownership and review process               | MIT license, contribution guide, governance, CI           |
| Security reviewers       | Visible trust boundaries and reporting           | Security policy, threat model, responsible disclosure     |

## 6. Functional Requirements

| ID    | Requirement | Description                                                                                       | Priority |
|-------|-------------|---------------------------------------------------------------------------------------------------|----------|
| FR-01 | CDN loading | The browser shall load Fyr from a versioned CDN URL without framework installation.               | Must     |
| FR-02 | Auto-start  | The core shall initialize after DOM readiness and mount declared Fyr application roots.           | Must     |
| FR-03 | Controllers | Developers shall register named controllers containing state, methods, computed values, watchers, | Must     |

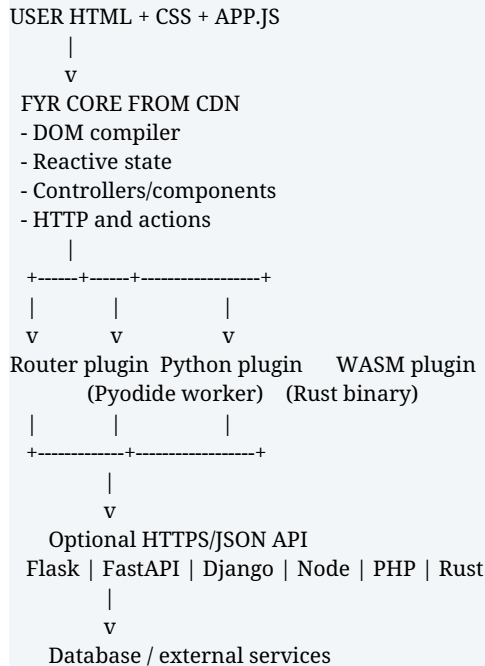
| ID    | Requirement     | Description                                                                                           | Priority |
|-------|-----------------|-------------------------------------------------------------------------------------------------------|----------|
|       |                 | and lifecycle hooks.                                                                                  |          |
| FR-04 | Reactive state  | State changes shall update dependent DOM bindings.                                                    | Must     |
| FR-05 | Text binding    | fyr-text shall render escaped text values.                                                            | Must     |
| FR-06 | Two-way binding | fyr-model shall synchronize form controls and state.                                                  | Must     |
| FR-07 | Events          | fyr-click and fyr-on:event shall execute permitted expressions or controller methods.                 | Must     |
| FR-08 | Conditions      | fyr-show and fyr-if shall support conditional display and rendering.                                  | Must     |
| FR-09 | Loops           | fyr-for shall repeat template content for iterable values.                                            | Must     |
| FR-10 | Forms           | fyr-submit shall prevent default navigation and execute controller logic.                             | Must     |
| FR-11 | Attributes      | fyr-bind, fyr-class, and fyr-style shall update DOM attributes and presentation.                      | Must     |
| FR-12 | Components      | Fyr shall support reusable HTML-first components with props, slots, and events.                       | Should   |
| FR-13 | HTTP            | Fyr.http shall provide GET, POST, PUT, PATCH, DELETE, timeout, error, cookie, and header handling.    | Must     |
| FR-14 | Actions         | Fyr.action shall invoke named server operations through a configurable endpoint.                      | Should   |
| FR-15 | Routing         | The router plugin shall support hash/history navigation, parameters, guards, and lazy page fragments. | Should   |
| FR-16 | Python          | The Python plugin shall lazy-load Pyodide and execute code in a worker.                               | Should   |
| FR-17 | WASM            | The WASM plugin shall load, cache, register, and call precompiled modules.                            | Should   |
| FR-18 | Storage         | The core shall expose localStorage, sessionStorage, and IndexedDB helpers.                            | Should   |
| FR-19 | Plugins         | Third parties shall be able to register directives, lifecycle hooks, and                              | Should   |

| ID    | Requirement | Description                                                                          | Priority |
|-------|-------------|--------------------------------------------------------------------------------------|----------|
|       |             | services.                                                                            |          |
| FR-20 | Diagnostics | Development mode shall provide useful errors for unknown directives and expressions. | Must     |

## 7. Non-Functional Requirements

| Area                   | Requirement                                                                       | Measure                                                                               |
|------------------------|-----------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| Usability              | A new user should create a counter using only HTML and one small JavaScript file. | First working page within 10 minutes using documentation.                             |
| Performance            | The core shall remain compact and avoid loading optional runtimes.                | Target minified+compressed core under 35 KB for 1.0, subject to benchmark validation. |
| Compatibility          | Support current evergreen browsers.                                               | Latest two stable versions of Chrome, Edge, Firefox, and Safari.                      |
| Reliability            | Pinned CDN versions must remain immutable.                                        | Published release assets verified with checksums.                                     |
| Security               | Expressions and HTML operations must minimize injection risks.                    | No eval/Function in hardened 1.0 expression engine; security tests pass.              |
| Accessibility          | Framework behavior must preserve semantic HTML and keyboard access.               | WCAG 2.2 AA-compatible patterns in official examples.                                 |
| Maintainability        | Modules must have clear boundaries and automated tests.                           | Coverage target at least 85% for core logic.                                          |
| Portability            | Applications should run on static hosting or with any backend.                    | Examples deploy on GitHub Pages, Netlify, Vercel, Render, and local servers.          |
| Transparency           | Users must understand what code executes.                                         | Public repository, source maps, changelog, threat model, and license.                 |
| Backward compatibility | Minor releases should not break documented APIs.                                  | Semantic Versioning and deprecation policy.                                           |

## 8. System Architecture



### 8.1 Architectural Principles

- CDN-only for users: no mandatory package installation or build step.
- Progressive enhancement: valid HTML remains the foundation.
- Modularity: optional features are separate files.
- Backend independence: Fyr communicates through web standards.
- Secure boundary: browser code is always considered inspectable and untrusted.
- Version stability: applications reference explicit versions instead of moving latest URLs.
- Extensibility: plugins may add directives and services without modifying the core.

## 9. Core Framework Design

### 9.1 Application Startup

On `DOMContentLoaded`, Fyr scans for `fyr-app` and `fyr-controller` roots, creates controller instances, wraps their state, compiles descendant directives, registers cleanup handlers, and runs mounted lifecycle hooks. `MutationObserver` support may compile HTML inserted after startup.

### 9.2 Reactive State

State is wrapped using JavaScript Proxy. Reads performed during an active effect register dependencies in a `WeakMap`. Writes notify only effects that depend on the changed object and key. A microtask scheduler batches repeated writes into one update cycle.



```

const targetMap = new WeakMap();
let activeEffect = null;

function track(target, key) {
  if (!activeEffect) return;
  // Store activeEffect under target and key.
}

function trigger(target, key) {
  // Queue only effects registered for this key.
}

```

### 9.3 Directive Model

| Directive      | Purpose                        | Example                               |
|----------------|--------------------------------|---------------------------------------|
| fyr-app        | Declare application root       | fyr-app="store"                       |
| fyr-controller | Attach state and methods       | fyr-controller="cart"                 |
| fyr-text       | Render escaped text            | fyr-text="user.name"                  |
| fyr-html       | Render explicitly trusted HTML | fyr-html="trustedContent"             |
| fyr-model      | Two-way form binding           | fyr-model="form.email"                |
| fyr-click      | Handle click                   | fyr-click="save()"                    |
| fyr-on:event   | Handle any supported event     | fyr-on:input="search()"               |
| fyr-show       | Toggle visibility              | fyr-show="loggedIn"                   |
| fyr-if         | Create/remove content          | fyr-if="items.length"                 |
| fyr-for        | Repeat template                | fyr-for="item in items"               |
| fyr-submit     | Handle form submit             | fyr-submit="login()"                  |
| fyr-bind:*     | Bind attribute/property        | fyr-bind:disabled="busy"              |
| fyr-class      | Bind CSS classes               | fyr-class="{ active: selected }"      |
| fyr-style      | Bind styles                    | fyr-style="{ width: progress + `%` }" |
| fyr-ref        | Store DOM reference            | fyr-ref="emailInput"                  |
| fyr-init       | Run startup method             | fyr-init="load()"                     |
| fyr-cloak      | Hide before initialization     | fyr-cloak                             |
| fyr-transition | Apply transition hooks         | fyr-transition="fade"                 |

### 9.4 Expression Safety

The prototype may initially use a restricted evaluator for rapid development, but a public hardened release must use a parser rather than JavaScript eval or Function. The parser should support literals, paths, arithmetic, comparisons, logical operators, arrays, objects, and calls to methods already exposed by the controller. It must reject global object access, constructors, prototype traversal, dynamic import, and arbitrary code creation.

### 9.5 Components

Components should remain HTML-first. A component definition contains a template or template URL, props, local state, methods, emitted events, and lifecycle hooks. Scoped state prevents two component instances from sharing mutable data accidentally.

## 10. CDN-Only Distribution Model

Fyr's defining product decision is that framework consumers do not install it. Versioned release files are published to npm only as a distribution mechanism, allowing jsDelivr and UNPKG to serve them globally. Users may also self-host the same dist directory.

| Asset                      | Loaded when                 | Purpose                                                        |
|----------------------------|-----------------------------|----------------------------------------------------------------|
| fyr.min.js                 | Every Fyr application       | Core state, directives, controllers, components, HTTP, actions |
| fyr-router.min.js          | Only routed applications    | Client-side routing and route guards                           |
| fyr-python.min.js          | Only browser-Python pages   | Pyodide loader, worker bridge, result conversion               |
| fyr-python.worker.js       | Loaded by Python plugin     | Runs Python off the main UI thread                             |
| fyr-wasm.min.js            | Only WASM applications      | Module loader, registry, memory helpers                        |
| fyr-socket.min.js          | Only real-time applications | WebSocket connection and reconnection                          |
| fyr-ui.min.js / fyr-ui.css | Only optional UI helpers    | Toast, modal, dialog, loading indicators                       |
| fyr-engine.wasm            | Only example/utility calls  | Precompiled Rust engine                                        |

## 10.1 Version Pinning

```
<script defer
src="https://cdn.jsdelivr.net/npm/fyr-framework@0.1.0/dist/fyr.min.js"
integrity="sha384-RELEASE_HASH"
crossorigin="anonymous"></script>
```

Documentation should recommend exact versions and Subresource Integrity where supported. The alias “latest” may be useful for experiments but should not appear in production examples.

## 10.2 Offline and Self-Hosted Use

CDN-only means no installation is required; it does not mean the application must always depend on a third-party network. Users may download a release asset once and serve it locally. A future service worker can cache the framework for offline use.

# 11. Optional Browser Python

The Python plugin uses Pyodide, which runs CPython compiled to WebAssembly. The plugin is intentionally separate because the Python runtime is much larger than the Fyr core. It initializes only after `Fyr.python.run` or `Fyr.python.loadPackage` is called.

```
<script defer src="../../../fyr.min.js"></script>
<script defer src="../../../fyr-python.min.js"></script>

<script>
async function calculate() {
  const result = await Fyr.python.run(
    values = [10, 20, 30]
    sum(values) / len(values)
  );
  console.log(result);
}
</script>
```

## 11.1 Appropriate Uses

- Programming education
- Mathematics and statistics
- Scientific demonstrations
- Local data transformation
- Offline calculations

- Interactive notebooks and coding exercises

## 11.2 Inappropriate Uses

- Passwords and API secrets
- Trusted authentication decisions
- Payment verification
- Unrestricted code from unknown users on the main page
- Direct privileged database access

## 11.3 Worker Controls

The plugin should support execution timeouts, worker termination, output size limits, package allowlists, memory guidance, and structured stdout/stderr capture. Running code in a worker improves responsiveness but does not create a perfect security sandbox.

## 12. Rust WebAssembly Integration

Rust source is compiled by framework or application maintainers before publication. End users load only the .wasm binary and Fyr's loader; they do not install Rust.

```
await Fyr.wasm.load(
  "math",
  "https://cdn.example.com/math.wasm"
);

const answer = Fyr.wasm.call("math", "add", 20, 22);
```

### 12.1 Suitable Workloads

- Image processing
- Compression and encoding
- Parsers and interpreters
- Simulation and game logic
- Audio processing
- Cryptographic primitives from audited libraries
- Large numeric loops

### 12.2 ABI and Memory Design

Simple numeric exports can be called directly. Strings, byte arrays, and objects require a documented application binary interface. Fyr should provide helpers for allocating memory, copying ArrayBuffer data, decoding UTF-8 strings, freeing buffers, and converting error codes into JavaScript exceptions.

## 13. Backend Independence

Fyr is a browser framework, not a Flask framework or FastAPI framework. Any backend that accepts HTTP requests and returns JSON can be used. This prevents vendor lock-in and allows users to select technology based on deployment, performance, team skill, or existing systems.

| Backend | Best fit                                     | Fyr connection                                              |
|---------|----------------------------------------------|-------------------------------------------------------------|
| Flask   | Beginners, traditional sites, small services | Fyr.http calls Flask JSON routes; Jinja may serve the page. |
| FastAPI | Typed APIs and async services                | Fyr.http or Fyr.action calls OpenAPI-backed endpoints.      |
| Django  | Large database-driven systems                | Django views/DRF return JSON; CSRF integration documented.  |

| Backend      | Best fit                                       | Fyr connection                                        |
|--------------|------------------------------------------------|-------------------------------------------------------|
| Node/Express | JavaScript teams and serverless APIs           | Standard fetch-compatible JSON endpoints.             |
| PHP/Laravel  | Shared hosting and PHP applications            | Normal HTTP forms or JSON APIs.                       |
| Rust/Axum    | High-throughput services                       | JSON endpoints and WebSocket services.                |
| No backend   | Static calculators, portfolios, learning tools | Everything runs in the browser; no protected storage. |

### 13.1 Flask Example

```

from flask import Flask, jsonify, request

app = Flask(__name__)

@app.get("/api/tasks")
def tasks():
    return jsonify({"items": []})

@app.post("/api/tasks")
def create_task():
    data = request.get_json() or {}
    return jsonify({"id": 1, "title": data.get("title")}), 201

const response = await Fyr.http.post("/api/tasks", {
  title: this.form.title
});

```

## 14. Security and Threat Model

All JavaScript, Python, and WebAssembly downloaded by the browser is visible and controllable by the user. Fyr must never imply that client-side code can protect secrets. The server remains responsible for authentication, authorization, payments, database credentials, sensitive validation, and audit records.

| Threat                  | Risk                                                 | Mitigation                                                                                     |
|-------------------------|------------------------------------------------------|------------------------------------------------------------------------------------------------|
| Cross-site scripting    | Untrusted content executes in the page               | fyr-text escapes content; fyr-html requires explicit trust; CSP guidance; parser restrictions. |
| Expression escape       | Directive expression accesses global objects         | Dedicated AST parser; allowlisted syntax; reject constructor/prototype/global traversal.       |
| CDN supply-chain attack | Distributed script is modified                       | Immutable versions, SRI, signed releases, protected publishing, reproducible builds.           |
| CSRF                    | Cookie-authenticated API is called from another site | SameSite cookies, CSRF tokens, Origin checks, backend protection.                              |
| Credential exposure     | Secret placed in app.js or browser Python            | Documentation and diagnostics; secrets only in server environment variables.                   |
| Malicious Python        | User code consumes CPU/memory                        | Worker isolation, timeout, terminate, size limits, allowlist packages.                         |
| Malicious WASM          | Module performs unexpected operations                | Load only trusted modules; integrity checks; limited imports; worker option.                   |
| Prototype pollution     | Attacker writes dangerous object paths               | Safe path parser; reject __proto__, constructor, and prototype keys.                           |

| Threat                | Risk                          | Mitigation                                                                         |
|-----------------------|-------------------------------|------------------------------------------------------------------------------------|
| Dependency compromise | Build dependency is malicious | Lockfiles, Dependabot, audit, minimal dependencies, review and SBOM.               |
| Denial of service     | Repeated API or runtime calls | Client throttling is advisory; backend rate limiting and quotas are authoritative. |

## 14.1 Responsible Disclosure

SECURITY.md should define supported versions, a private reporting channel, expected acknowledgement time, coordinated disclosure, and credit rules. Public issues must not be used for unpatched vulnerabilities.

## 15. Data Structures and Algorithms

| Component           | Data structure / algorithm                    | Reason                                                          |
|---------------------|-----------------------------------------------|-----------------------------------------------------------------|
| Dependency tracking | WeakMap<object, Map<key, Set<effect>>>        | Garbage-collectable object keys and efficient targeted updates. |
| Controller registry | Map<string, definition>                       | Fast lookup by controller name.                                 |
| WASM registry       | Map<string, WebAssembly.Instance>             | Named module access and reuse.                                  |
| Directive registry  | Map<string, handler>                          | Plugin-defined directive dispatch.                              |
| Effect queue        | Set<effect> plus microtask                    | Deduplicates repeated updates in one event loop turn.           |
| Template loop       | Keyed reconciliation map                      | Reuses DOM nodes where keys remain stable.                      |
| Router              | Route tree or compiled regular expressions    | Matches paths and extracts parameters.                          |
| HTTP retry          | Exponential backoff with jitter               | Avoids synchronized retry bursts.                               |
| Cache               | LRU or timestamp map                          | Limits cached fragments/modules.                                |
| Expression parser   | Tokenizer + Pratt parser or recursive descent | Evaluates a safe subset without executing arbitrary JavaScript. |

### 15.1 Update Complexity

With dependency tracking, changing one state property should notify only effects registered for that key. Lookup is approximately  $O(1)$  for target and key maps, followed by  $O(k)$  scheduling where  $k$  is the number of dependent effects. A full DOM rescan after every change would instead approach  $O(n)$ , where  $n$  is the number of compiled nodes, and is therefore avoided.

### 15.2 Keyed List Reconciliation

fyr-for should support optional keys. The algorithm maps previous keys to DOM blocks, reuses blocks found in the new sequence, creates missing blocks, updates scope values, reorders nodes, and removes obsolete blocks. This reduces unnecessary destruction and preserves input focus and component state.

## 16. User Experience and Accessibility

- Fyr should preserve semantic HTML rather than replacing buttons, links, labels, and headings with custom non-semantic elements.
- Official components must support keyboard operation, focus visibility, labels, error association, and reduced-motion preferences.
- fyr-cloak prevents uninitialized template flashes but must not hide content permanently when JavaScript fails.

- Loading and error states should be represented in controller state and announced with appropriate ARIA live regions.
- Documentation examples should use responsive layouts and avoid color-only status indicators.
- The framework documentation website should meet WCAG 2.2 AA targets.

## 17. Testing and Quality Assurance

| Test level           | Tools / method                               | Coverage                                                    |
|----------------------|----------------------------------------------|-------------------------------------------------------------|
| Unit                 | Vitest + jsdom/happy-dom                     | Reactivity, parser, directives, HTTP, router, utilities     |
| Browser integration  | Playwright                                   | DOM updates, forms, navigation, workers, CDN builds         |
| Cross-browser        | Playwright projects and manual Safari checks | Chrome, Edge, Firefox, WebKit                               |
| Python plugin        | Browser worker tests with Pyodide            | Initialization, output, errors, timeout, package loading    |
| WASM plugin          | Node and browser tests                       | Load, instantiate, call, memory transfer, cache             |
| Backend examples     | pytest / framework test clients              | Flask/FastAPI authentication, CRUD, CORS, CSRF              |
| Security             | Dedicated adversarial tests                  | XSS, parser escape, prototype pollution, unsafe URL schemes |
| Accessibility        | axe-core plus keyboard review                | Official examples and docs                                  |
| Release verification | Checksum and smoke test against CDN artifact | Published files match build output                          |

### 17.1 Definition of Done

- Feature documented
- Unit tests added
- Browser test added where relevant
- Security impact reviewed
- Bundle size checked
- No unexplained console errors
- Changelog entry included
- Backward compatibility considered

## 18. Performance and Benchmarking

- Measure compressed transfer size for each CDN asset.
- Measure startup time for 100, 1,000, and 10,000 bindings.
- Measure keyed list insertion, removal, and reordering.
- Measure memory after repeated component mount and destroy cycles.
- Measure Pyodide cold start separately from warm execution.
- Measure WASM cold instantiation and repeated call overhead.
- Compare representative tasks against plain JavaScript and similar CDN-oriented frameworks.
- Publish raw benchmark code and environment details to avoid misleading claims.

**Important:** Performance claims must be based on reproducible measurements. The report sets targets, not unverified marketing statements.

## 19. Open-Source Governance

### 19.1 License

The recommended license is MIT because it is short, widely understood, permissive, and compatible with commercial and educational use. Third-party dependencies retain their own licenses and must be documented in a notices file or generated software bill of materials.

### 19.2 Public Repository Files

- LICENSE
- README.md
- CONTRIBUTING.md
- CODE\_OF\_CONDUCT.md
- SECURITY.md
- GOVERNANCE.md
- CHANGELOG.md
- ROADMAP.md
- SUPPORT.md
- Third-party notices and SBOM

### 19.3 Contribution Workflow

1. Contributor opens or selects an issue.
2. Maintainers confirm scope and design direction.
3. Contributor creates a focused branch and tests.
4. Pull request explains behavior, security impact, tests, and documentation.
5. Automated CI checks formatting, types, unit tests, browser tests, size, and security scans.
6. At least one maintainer reviews; high-risk parser or security changes require two reviewers.
7. Change is merged using a consistent strategy and included in the changelog.

### 19.4 Roles

| Role                     | Responsibility                                               |
|--------------------------|--------------------------------------------------------------|
| Project lead             | Vision, final architectural decisions, release authorization |
| Core maintainer          | Reactivity, compiler, directives, component system           |
| Runtime maintainer       | Python and WebAssembly plugins                               |
| Documentation maintainer | Guides, API reference, examples, migration notes             |
| Security maintainer      | Threat model, disclosures, advisories, security tests        |
| Contributor              | Issues, code, tests, documentation, translations, feedback   |

### 19.5 Versioning and Deprecation

Fyr should use Semantic Versioning. Patch releases fix compatible defects, minor releases add compatible features, and major releases may remove deprecated APIs. A documented API should normally remain deprecated for at least one minor release before removal, except when an immediate security fix is necessary.

## 20. Repository and File Structure

The repository contains development source and release assets. Users interact only with dist files; contributors work with the full source tree.

```

fjr-framework/
├── README.md
├── LICENSE
├── CHANGELOG.md
├── CONTRIBUTING.md
├── CODE_OF_CONDUCT.md
├── SECURITY.md
├── GOVERNANCE.md
├── ROADMAP.md
├── SUPPORT.md
├── package.json
├── tsconfig.json
├── vite.config.ts
├── vitest.config.ts
├── playwright.config.ts
├── eslint.config.js
├── prettier.config.js
├── .editorconfig
├── .gitignore
├── .npmrc
├── src/
│   ├── core/
│   │   ├── index.ts
│   │   ├── fjf.ts
│   │   └── app/
│   │       ├── create-app.ts
│   │       ├── mount-app.ts
│   │       ├── destroy-app.ts
│   │       └── app-registry.ts
│   │   └── reactivity/
│   │       ├── reactive.ts
│   │       ├── effect.ts
│   │       ├── computed.ts
│   │       ├── watch.ts
│   │       ├── scheduler.ts
│   │       └── dependency-map.ts
│   │   └── compiler/
│   │       ├── dom-scanner.ts
│   │       ├── node-compiler.ts
│   │       ├── directive-parser.ts
│   │       ├── expression-tokenizer.ts
│   │       ├── expression-parser.ts
│   │       ├── expression-evaluator.ts
│   │       ├── scope.ts
│   │       └── cleanup.ts
│   │   └── controllers/
│   │       ├── controller.ts
│   │       ├── registry.ts
│   │       ├── instance.ts
│   │       └── lifecycle.ts
│   │   └── directives/
│   │       ├── registry.ts
│   │       ├── fjf-text.ts
│   │       ├── fjf-html.ts
│   │       ├── fjf-model.ts
│   │       ├── fjf-click.ts
│   │       ├── fjf-on.ts
│   │       ├── fjf-show.ts
│   │       ├── fjf-if.ts
│   │       ├── fjf-for.ts
│   │       ├── fjf-submit.ts
│   │       └── fjf-bind.ts

```



```

├── fyr-class.ts
├── fyr-style.ts
├── fyr-ref.ts
├── fyr-init.ts
├── fyr-cloak.ts
├── fyr-transition.ts
├── components/
│   ├── define-component.ts
│   ├── registry.ts
│   ├── instance.ts
│   ├── props.ts
│   ├── slots.ts
│   └── events.ts
├── http/
│   ├── client.ts
│   ├── interceptors.ts
│   ├── errors.ts
│   ├── retry.ts
│   ├── timeout.ts
│   └── csrf.ts
├── actions/
├── plugins/
├── storage/
├── events/
├── security/
├── utilities/
├── types/
├── router/
├── python/
│   ├── pyodide-loader.ts
│   ├── python-runtime.ts
│   ├── python.worker.ts
│   ├── package-loader.ts
│   ├── javascript-bridge.ts
│   └── security-policy.ts
├── wasm/
│   ├── wasm-loader.ts
│   ├── wasm-registry.ts
│   ├── memory-manager.ts
│   └── wasm-errors.ts
├── socket/
├── ui/
├── rust/
│   ├── Cargo.toml
│   └── src/
│       ├── lib.rs
│       ├── memory.rs
│       ├── strings.rs
│       └── arrays.rs
├── dist/
│   ├── fyr.js
│   ├── fyr.min.js
│   ├── fyr.esm.js
│   ├── fyr-router.min.js
│   ├── fyr-python.min.js
│   ├── fyr-python.worker.js
│   ├── fyr-wasm.min.js
│   ├── fyr-socket.min.js
│   ├── fyr-ui.min.js
│   └── fyr-ui.css

```

```

├── fyr-engine.wasm
├── *.map
├── checksums.txt
├── examples/
│   ├── counter/
│   ├── todo/
│   ├── router/
│   ├── browser-python/
│   ├── rust-wasm/
│   ├── flask-api/
│   ├── fastapi-api/
│   ├── ecommerce/
│   └── realtime-chat/
├── tests/
│   ├── unit/
│   ├── integration/
│   ├── e2e/
│   ├── security/
│   ├── accessibility/
│   └── fixtures/
├── docs/
│   ├── getting-started/
│   ├── guide/
│   ├── api/
│   ├── backends/
│   ├── security/
│   ├── contributing/
│   └── examples/
├── benchmarks/
├── scripts/
│   ├── build.mjs
│   ├── test.mjs
│   ├── release.mjs
│   ├── verify-dist.mjs
│   ├── generate-sri.mjs
│   └── build-wasm.ps1
├── .github/
│   ├── ISSUE_TEMPLATE/
│   ├── PULL_REQUEST_TEMPLATE.md
│   ├── dependabot.yml
│   ├── CODEOWNERS
│   └── workflows/
│       ├── ci.yml
│       ├── security.yml
│       ├── size.yml
│       ├── release.yml
│       └── deploy-docs.yml

```

## 21. Development Methodology

An iterative and test-driven approach is suitable. Each phase should produce a usable increment rather than building all optional modules simultaneously.

| Phase         | Deliverable                                                        |
|---------------|--------------------------------------------------------------------|
| 1. Foundation | Repository, license, standards, build output, basic CDN smoke page |
| 2. Reactivity | Proxy engine, effects, scheduler, computed values,                 |

| Phase                     | Deliverable                                                              |
|---------------------------|--------------------------------------------------------------------------|
|                           | watchers                                                                 |
| 3. Compiler               | DOM scanning, scope, safe expressions, cleanup                           |
| 4. Directives             | Core directive set with unit and browser tests                           |
| 5. Controllers/components | Lifecycle, component props, slots, events                                |
| 6. Networking             | HTTP client, errors, actions, CSRF hooks                                 |
| 7. Router                 | Hash/history navigation, params, guards, lazy fragments                  |
| 8. Python                 | Pyodide lazy loader, worker, package rules, timeout                      |
| 9. WASM                   | Module registry, memory bridge, Rust examples                            |
| 10. Hardening             | Security parser, CSP compatibility, a11y, leak tests                     |
| 11. Documentation         | Guides, API reference, examples, migration policy                        |
| 12. Release               | Signed/tagged release, checksums, SRI, npm publication, CDN verification |

## 22. Deployment and Release Strategy

8. Run linting, type checking, unit, integration, browser, accessibility, and security tests.
9. Build unminified, minified, ESM, worker, CSS, source-map, and WebAssembly assets.
10. Verify no development-only code or secrets exist in dist.
11. Generate SHA-256 checksums and SRI hashes.
12. Create a version tag following Semantic Versioning.
13. Publish the package to npm with dist entry fields.
14. Verify jsDelivr and UNPKG URLs return the exact release files.
15. Publish documentation and release notes.
16. Run a smoke application using the public CDN URLs.
17. Monitor issue reports and prepare security advisories where needed.

### 22.1 User Deployment

A basic Fyr application consists of static files and may be hosted on GitHub Pages, Netlify, Vercel static hosting, Cloudflare Pages, an object-storage website, Apache, Nginx, or a Flask/Django template directory. A backend is required only for protected data or server operations.

## 23. Risks, Limitations, and Mitigation

| Risk / Limitation                          | Impact                                       | Mitigation                                                                       |
|--------------------------------------------|----------------------------------------------|----------------------------------------------------------------------------------|
| CDN unavailable                            | Application cannot download uncached scripts | Allow self-hosting, service-worker cache, and fallback URL guidance.             |
| Pyodide is large                           | Slow first browser-Python use                | Separate plugin, lazy load, progress state, cache, worker.                       |
| Expression parser complexity               | Security bugs or inconsistent syntax         | Small grammar, fuzz tests, security review, avoid full JavaScript compatibility. |
| Framework scope grows too quickly          | Maintenance becomes difficult                | Small core, optional plugins, milestone discipline.                              |
| One-person maintainer risk                 | Slow reviews and releases                    | Governance, documented processes, recruit maintainers.                           |
| Backend examples mistaken for requirements | Users think Flask/FastAPI is mandatory       | Clearly label backend independence throughout docs.                              |
| WASM bridge complexity                     | Memory leaks or incorrect conversions        | Documented ABI, wrappers, tests, explicit free functions.                        |

| Risk / limitation        | Impact                                         | Mitigation                                                                 |
|--------------------------|------------------------------------------------|----------------------------------------------------------------------------|
| Browser differences      | Features behave inconsistently                 | Compatibility matrix and cross-browser CI.                                 |
| Open-source supply chain | Publisher account or dependency compromise     | 2FA, protected releases, minimal dependencies, SRI, provenance.            |
| No virtual DOM           | Certain huge dynamic UIs may be less efficient | Targeted effects and keyed reconciliation; document appropriate use cases. |

## 24. Evaluation Criteria

| Criterion              | Success indicator                                                                         |
|------------------------|-------------------------------------------------------------------------------------------|
| Zero-install usage     | A user creates and runs an application using only a script tag and static files.          |
| Reactive correctness   | DOM stays synchronized after object, array, and nested state changes.                     |
| Directive completeness | All version-1 directives pass unit and browser tests.                                     |
| Backend independence   | Same frontend works against at least Flask and FastAPI examples with minimal URL changes. |
| Plugin isolation       | Core works when Python, WASM, router, socket, and UI modules are absent.                  |
| Security               | Safe parser blocks documented escape attempts; XSS and prototype-pollution tests pass.    |
| Performance            | Published benchmark suite demonstrates acceptable startup and update behavior.            |
| Accessibility          | Official examples pass automated checks and keyboard review.                              |
| Open-source readiness  | License, governance, contribution, security, and release documents are public.            |
| Documentation          | A beginner completes counter, todo, API, Python, and WASM tutorials.                      |

## 25. Roadmap

| Release | Primary goals                                                                               |
|---------|---------------------------------------------------------------------------------------------|
| 0.1.0   | Core reactive state, controllers, basic directives, HTTP, CDN build, counter/todo examples. |
| 0.2.0   | Safe expression parser, computed/watch, improved errors, keyed fyr-for.                     |
| 0.3.0   | Components, props, slots, events, lifecycle cleanup.                                        |
| 0.4.0   | Router, route guards, lazy HTML fragments.                                                  |
| 0.5.0   | Flask and FastAPI reference kits, server actions, CSRF documentation.                       |
| 0.6.0   | Python plugin with worker, timeout, package controls.                                       |
| 0.7.0   | WASM registry, memory helpers, Rust examples.                                               |
| 0.8.0   | Socket plugin, offline caching experiment, dev diagnostics.                                 |
| 0.9.0   | Security audit, accessibility review, benchmark publication, migration guide.               |
| 1.0.0   | Stable API, production documentation, signed releases, compatibility policy.                |

## 26. Conclusion

Fyr addresses a real educational and practical problem: web applications should not always require a large installation and build process before meaningful interactivity is possible. Its CDN-only delivery allows developers to begin with ordinary HTML while retaining a path to reactive state, components, routing, remote APIs, browser Python, and Rust WebAssembly.

The open-source model strengthens the project by making implementation decisions, security limitations, test results, and release artifacts visible to the community. The most important design discipline is maintaining a small and secure core. Flask, FastAPI, Python, Rust, UI helpers, routing, and sockets are integrations or optional modules, not mandatory weight imposed on every application.

A successful Fyr 1.0 release will therefore be measured not by the number of features it contains, but by whether users can trust its CDN assets, understand its HTML-first programming model, extend it safely, connect it to the backend of their choice, and contribute to its development through a clear open-source process.

## Appendix A: Minimum User Project

```
my-fyr-app/
├─ index.html
├─ app.js
└─ styles.css
```

## Appendix B: Full CDN Example

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Fyr Todo</title>
  <link rel="stylesheet" href="styles.css">
  <script defer src="https://cdn.jsdelivr.net/npm/fyr-framework@0.1.0/dist/fyr.min.js"></script>
  <script defer src="app.js"></script>
</head>
<body>
  <main fyr-controller="todo">
    <form fyr-submit="addTask()">
      <input fyr-model="draft" required>
      <button fyr-bind:disabled="!draft.trim()">Add</button>
    </form>

    <template fyr-for="task in tasks" fyr-key="task.id">
      <article>
        <input type="checkbox" fyr-model="task.done">
        <span fyr-text="task.title"></span>
        <button fyr-click="removeTask(task.id)">Delete</button>
      </article>
    </template>
  </main>
</body>
</html>
```

```

Fyr.controller("todo", {
  state: {
    draft: "",
    tasks: []
  },
  methods: {
    addTask() {
      const title = this.draft.trim();
      if (!title) return;
      this.tasks.push({ id: crypto.randomUUID(), title, done: false });
      this.draft = "";
    },
    removeTask(id) {
      this.tasks = this.tasks.filter(task => task.id !== id);
    }
  }
});

```

## Appendix C: Recommended Public Repository Documents

| Document                | Purpose                                                |
|-------------------------|--------------------------------------------------------|
| README.md               | Quick start, CDN links, features, examples, status     |
| LICENSE                 | MIT permissions and conditions                         |
| CONTRIBUTING.md         | Development setup, style, tests, pull requests         |
| CODE_OF_CONDUCT.md      | Community behavior and enforcement                     |
| SECURITY.md             | Supported versions and private vulnerability reporting |
| GOVERNANCE.md           | Roles, decisions, maintainers, succession              |
| CHANGELOG.md            | Human-readable release changes                         |
| ROADMAP.md              | Planned milestones without guarantees                  |
| SUPPORT.md              | Where to ask questions and what support is available   |
| NOTICE / THIRD_PARTY.md | Third-party licenses and attributions                  |

## Appendix D: Glossary

| Term                    | Meaning                                                                                         |
|-------------------------|-------------------------------------------------------------------------------------------------|
| CDN                     | A distributed network that serves static files from locations close to users.                   |
| Directive               | A Fyr-specific HTML attribute that connects an element to state or behavior.                    |
| Reactive state          | Data that automatically updates dependent interface bindings when changed.                      |
| Controller              | A named object containing state, methods, computed values, watchers, and lifecycle hooks.       |
| Pyodide                 | CPython compiled to WebAssembly for execution in a browser.                                     |
| WebAssembly             | A compact binary instruction format supported by modern browsers.                               |
| Server action           | A named backend operation invoked by the browser through an HTTP endpoint.                      |
| SRI                     | Subresource Integrity, which lets a browser verify a fetched file against a cryptographic hash. |
| Semantic Versioning     | Version format MAJOR.MINOR.PATCH based on compatibility.                                        |
| Progressive enhancement | Starting with functional HTML and adding richer behavior                                        |

| Term | Meaning                     |
|------|-----------------------------|
|      | when scripts are available. |